EFFICIENT IMPLEMENTATION OF TRIDENDRIFORM SCHROEDER TREE ALGEBRA

P. CATOIRE, J. FROMENTIN

Abstract. We introduce a primitive computation problem in the free tridendriform algebra generated by one element which is a Hopf algebra based on Schroeder trees. We know a complex way to generate all of them, hence we want to implement this method on a computer. However, we need to create some tools to implement Schroeder trees and the multiplications over this algebra to be able to compute the primitive elements. We also checked numerically that they are all primitive elements. In this paper, we detail how we made the problem mathematically understandable for a computer and how we implement it.

1. Introduction

1.1. **Aim and structure of the paper.** The objective of this work is to implement the objects and operations of a Hopf algebra involving Schroeder trees. Those trees introduced in definition 1 can be endowed with three linear maps $\prec, \succ$ and $\cdot$ making it a free tridendriform algebra structure [11, 3, 18] where the products can be described with quasi-shuffles (theorem 1) . Tridendriform algebras arise in different topics in mathematics like free probability [6, 8], the study of Rota-Baxter algebras [17, 16] or in other topics like number theory [10, 4]. Some computation tools already exist for some algebras like Loday-Ronco algebra [7] on Sagemath [19, 1] but not for the free tridendriform algebra. Hence, implementing the free tridendriform algebra [3] (over one generator) allows us to implement any tridendriform algebras coding a quotient map by a tridendriform ideal from Schroeder trees to the desired objects.

This algebra turns out to be a Hopf algebra with its coproduct given in theorem 2. Computing the space of primitive elements of this algebra, we describe at least the largest cocommutative algebra inside it thanks to the Cartier-Quillen-Milnor-Moore theorem [12, 13, 2]. An inductive algorithm is given in the article [3] (shown in figure 1) to compute those elements using only $\prec, \succ$ and $\cdot$ operations. We also implement the coproduct of this algebra in terms of admissible cuts, see section 3.4.2.

Initially, we wanted to implement efficiently (i.e with low complexity) this algorithm to compute all primitive elements in any degree. But to our surprise, using some naive approaches, this task is really difficult. Hence, we need to choose a point of view of the problem to get a clear and efficient implementation of this algorithm.

To solve this problem, we used a particular representations of Schroeder trees as initial chains (definition 9) or specific sequences over the alphabet $\{0, 1\}$ (definition 13). Those representations are called *codes* of the tree. Thanks to this point of view, we are able to describe the action of quasi-shuffles on a pair of codes in definition 18 such that it describes the initial action of these elements onto a pair of Schroeder trees (theorem 1). We present the implementation in `C++` of the trees as codes and of the three basics operations $\prec, \succ$ and $\cdot$ on codes that are equivalent to the original ones on Schroeder trees.

The paper is divided in the following way:

- Section 2 describes and gives the minimum piece of information needed to understand the general mathematical setting in which we are working. We introduce the main objects

of this work, Schroeder trees in definition 1 and quasi-shuffles in definition 4. Then, we detail the Hopf algebra structure on $(\mathbb{K}\,\mathsf{Sch}, \prec, \succ, \cdot)$ (figure 1) and we introduce the algorithm which computes its space of primitive elements.

- Section 3 is a mathematical description of the encoding used to implement our problem. We introduce the concept of Schroeder levelled trees (definition 7), initial chains and the map associating to any initial chain a 0's and 1's sequence (definition 9). Then, we choose a way to map any Schroeder tree to a levelled Schroeder tree (proposition 1) which gives a way to get a 0's and 1's sequence from any Schroeder tree. A sequence obtained by this transformation will be called a *tree code*. Finally, we present a tridendriform structure on tree codes (definition 18) that turns out to be isomorphic to the one on Schroeder trees (corollary 1). We also give details how one finds admissible cuts from a tree code.
- The last section 4 explains the implementation we made in `C++` to perform some computations and the results it produces.

1.2. **Notations used in the article.** In this list, we put a table of contents of notations used here :

- for any $n \in \mathbb{N}^*$, $[\![1, n]\!]$ is the set $\{1, \dots, n\}$;
- for any set X, we denote by $\mathbb{K}\,X$ the vector space over $\mathbb{K}$ whose basis is X;
- for any set $X, X^\star$ denotes the set of finite words over the alphabet X. We denote by $T(\mathbb{K}\,X)$ the $\mathbb{K}$-vector space whose basis is $X^\star$;
- for any graded set $X = \bigoplus_{n=0}^{+\infty} X_n$, for any $n \in \mathbb{N}$, we define $T(\mathbb{K}\,X)(n)$ the subspace whose basis is the set $X^\star$ where the sum of the degree's of its letters is n;
- $(\mathbb{K}\,\mathsf{Sch}, \prec, \cdot, \succ)$ the graded tridendriform algebra on Schroeder tree;
- $\mathsf{Sch}(n)$ is the set of Schroeder tree with n leaves;
- FSch is the set of forest of Schroeder tree;
- $\vee$ is the grafting operator of many trees onto a common root;
- given a sequence $x = (x_1, \dots, x_n)$ and $i \in [\![1, n]\!]$, we denote by $x \setminus (x_i)$ the sequence x without the term x_i;
- for reasons of length some increasing sequences will be written in columns. The maximal element is at the top of the column representation of the sequence.

1.3. **Acknowledgments.** The authors would like to thank LMPA of ULCO for welcoming the first author during this collaboration.

2. About the original problem

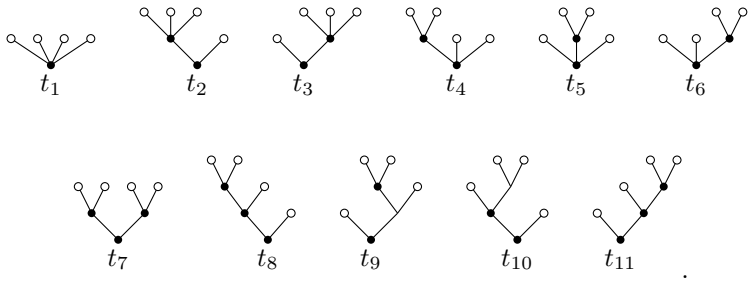
2.1. **Schroeder trees and comb representations.** Let us remind the following definition:

Definition 1. A *Schroeder tree* is a planar rooted tree such that any vertices which is not a leaf has at least 2 children. This set can be graded with the number of leaves minus 1. For any $n \in \mathbb{N} \setminus \{0\}$, we define $\mathsf{Sch}(n)$ the set of Schroeder trees with $n+1$ leaves. We also put $\mathsf{Sch}(0) = \{\circ\}$ and define:

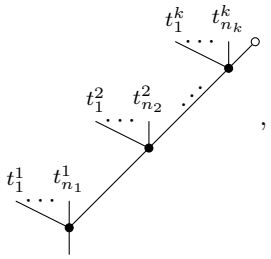
$$\mathsf{Sch} \coloneqq \bigcup_{n=0}^{+\infty} \mathsf{Sch}(n) \text{ and } \overline{\mathsf{Sch}} = \bigcup_{n=1}^{+\infty} \mathsf{Sch}(n).$$

Given any $t \in \mathsf{Sch}$, we will denote $V(t)$ the set of vertices which are not leaves and $E(t)$ its set of edges. We call them *internal vertices* of t. In the sequel, they are drawn as black disks.

Example 1. The elements of $\mathsf{Sch}(3)$ are

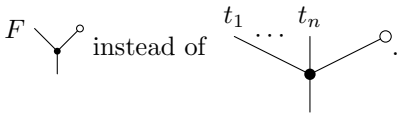


Let t be a tree. It can always be seen as a comb:

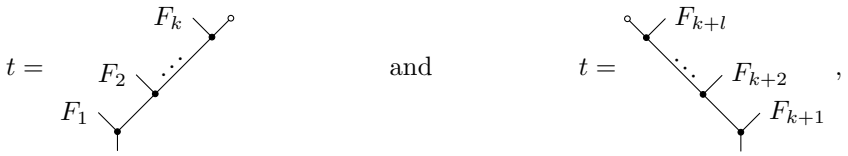


where k is the number of nodes on the right-most branch of t and for $i \in [\![1, k]\!], n_i + 1$ is the number of sons of the i-th node of this branch.

Notation 1. Let $F = t_1 \dots t_n$ be a forest composed of n trees. For simplicity, we write:



With these notations, we notice that all tree t can be seen either respectively like a *right comb* or a *left comb*:

(1) 

where $F_1, \dots, F_k$ and $F_{k+1}, \dots, F_{k+l}$ are non-empty forests.

Definition 2. The writing defined above at the right is the writing of t as a *right comb*. Proceeding the same way, looking at the left branch of the root of t, we get the writing of t as a *left comb*.

2.2. **Our objective.** We describe the structure of bialgebra onto $\mathbb{K}\,\mathsf{Sch}$ and we explain how to compute its primitive elements thanks to some results [3, 15].

Definition 3. Let $\mathbb{K}$ be a field and A be vector space of $\mathbb{K}$. We say $(A, \prec, \cdot, \succ)$ is a *tridendriform algebra* if $\prec, \cdot$ and $\succ$ are three linear maps from $A \otimes A$ to A such that for any $a, b, c \in A$:

$$(a \prec b) \prec c = a \prec (b * c), \tag{2}$$
$$(a \succ b) \prec c = a \succ (b \prec c), \tag{3}$$
$$(a * b) \succ c = a \succ (b \succ c), \tag{4}$$
$$(a \succ b) \cdot c = a \succ (b \cdot c), \tag{5}$$
$$(a \prec b) \cdot c = a \cdot (b \succ c), \tag{6}$$
$$(a \cdot b) \prec c = a \cdot (b \prec c), \tag{7}$$
$$(a \cdot b) \cdot c = a \cdot (b \cdot c), \tag{8}$$

where $*$ is the *associative product* of the tridendriform algebra defined for any $a, b \in A$ by $a * b \coloneqq a \prec b + a \cdot b + a \succ b$. We call the products $\succ, \prec, \cdot$ respectively right, left and middle.

The free tridendriform algebra is built on the vector space $\mathbb{K}\,\overline{\mathsf{Sch}}$. We then add a unit $\circ$ to the product $*$ by hand to get an algebra over $\mathbb{K}\,\mathsf{Sch} \oplus \mathbb{K} \cdot_\circ = \mathbb{K}\,\mathsf{Sch}$. We will denote it by $(\mathsf{Sch}, \prec, \cdot, \succ)$. We have a combinatorial interpretation of the product $*$. For this let us introduce:

Definition 4 (Quasi-shuffle)**.** Let $k, l \in \mathbb{N} \setminus \{0\}$. A (k,l)-*quasi-shuffle* is a surjective map $\sigma : [\![1, k+l]\!] \twoheadrightarrow [\![1, n]\!]$ for some positive integer n such that:

$$\sigma(1) < \cdots < \sigma(k) \text{ and } \sigma(k+1) < \cdots < \sigma(k+l).$$

We will denote $\mathrm{QSh}(k,l)$ the set of all (k,l)-quasi-shuffles.

We can easily prove that:

Lemma 1. *Let $k, l \geq 2$. Then, the following sets are in bijections:*

$$\{\sigma \in \mathrm{QSh}(k,l) \,|\, \sigma^{-1}(\{1\}) = \{1\}\} \simeq \mathrm{QSh}(k-1, l),$$
$$\{\sigma \in \mathrm{QSh}(k,l) \,|\, \sigma^{-1}(\{1\}) = \{k+1\}\} \simeq \mathrm{QSh}(k, l-1),$$
$$\{\sigma \in \mathrm{QSh}(k,l) \,|\, \sigma^{-1}(\{1\}) = \{1, k+1\}\} \simeq \mathrm{QSh}(k-1, l-1).$$

For other cases, we have:

$$\mathrm{QSh}(1,0) = \{\mathrm{Id}_{\{1\}}\} = \mathrm{QSh}(0,1) \text{ and } \mathrm{QSh}(1,1) = \{\mathrm{Id}_{\{1,2\}}, (2,1)\}.$$

Proof. Let k, l be two integers greater or equal to 2. Let $\sigma \in \mathrm{QSh}(k,l)$.

First case: $\sigma^{-1}(\{1\}) = \{1\}$. We denote by $\sigma' : [\![1, k+l-1]\!] \to \mathbb{N}$ the unique map such that:

$$\forall n \in [\![1, k+l]\!], \sigma(n) = \begin{cases} 1 & \text{if } n = 1, \\ \sigma'(n-1) + 1 & \text{otherwise.} \end{cases}$$

It is left to the reader to check that σ' is an element of $\mathrm{QSh}(k-1, l)$.

Second case: $\sigma^{-1}(\{1\}) = \{k+1\}$. We denote by $\sigma' : [\![1, k+l-1]\!] \to \mathbb{N}$ the unique map such that:

$$\forall n \in [\![1, k+l]\!], \sigma(n) = \begin{cases} 1 & \text{if } n = k+1, \\ \sigma'(n) + 1 & \text{if } n < k+1, \\ \sigma'(n-1) + 1 & \text{otherwise.} \end{cases}$$

It is left to the reader to check that σ' is an element of $\mathrm{QSh}(k, l-1)$.

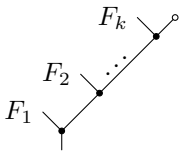
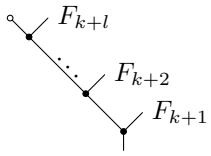
Third case: $\sigma^{-1}(\{1\}) = \{1, k+1\}$. We denote $\sigma' : [\![1, k+l-1]\!] \to \mathbb{N}$ the unique map such that:

$$\forall n \in [\![1, k+l]\!], \sigma(n) = \begin{cases} 1 & \text{if } n = 1 \text{ or } n = k+1, \\ \sigma'(n-1)+1 & \text{if } n \leq k, \\ \sigma'(n-2)+1 & \text{otherwise.} \end{cases}$$

It is left to the reader to check that σ' is an element of $\mathrm{QSh}(k-1, l-1)$.

Hence it establishes maps from the left sets to the right sets in the lemma. Conversely, one remarks in any case that the map $\sigma \mapsto \sigma'$ is invertible. □

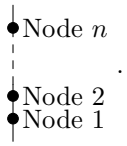
Let t, s two trees different from $\circ$. We look t as a right comb and s as a left comb. In other words, we put:

(9) $t =$  and $s =$  ,

where for all $i \in [\![1, k+l]\!]$, F_i is a non-empty forest. Here, k represents the number of nodes on the right-most branch of t and l is the number of nodes on the left-most branch of s.

Definition 5. Let t and s be two trees which respective right comb representation and left comb representation are given above (equation (9)). We denote by k (respectively l) the number of nodes on the right-most (respectively left-most) branch of t (respectively s). Let σ be a (k, l)-quasi-shuffle which has for image $[\![1, n]\!]$ with n a non-negative integer. We denote $\sigma(t, s)$ the tree obtained this way:

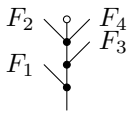
(1) We first consider the ladder with n nodes:



.

(2) For all $i \in [\![1, k]\!]$, we graft F_i as the *left* son at the node $\sigma(i)$.
(3) For all $i \in [\![k+1, k+l]\!]$, we graft F_i as *right* son at the node $\sigma(i)$.

Example 2. Consider the $(2, 2)$-quasi-shuffle $\sigma = (1, 3, 2, 3)$.

Let us take $t =$  and $s =$  . Then $\sigma(t, s) =$  .

Theorem 1 ([3])**.** *Let t, s be two trees different from $\circ$ as described above. Then:*

$$t * s = \sum_{\sigma \in \mathrm{QSh}(k,l)} \sigma(t, s).$$

Moreover:

$$t \prec s = \sum_{\substack{\sigma \in \mathrm{QSh}(k,l) \\ \sigma^{-1}(\{1\}) = \{1\}}} \sigma(t, s), t \succ s = \sum_{\substack{\sigma \in \mathrm{QSh}(k,l) \\ \sigma^{-1}(\{1\}) = \{k+1\}}} \sigma(t, s),$$

$$t \cdot s = \sum_{\substack{\sigma \in \mathrm{QSh}(k,l) \\ \sigma^{-1}(\{1\}) = \{1, k+1\}}} \sigma(t, s).$$

This result is a combinatorial description of an inductive formula given in the previous work of J.L. Loday and M. Ronco [11]. Surprisingly, $(\mathbb{K}\,\mathsf{Sch}, \prec, \cdot, \succ)$ can be endowed with a codendriform coproduct [3, 11] detailled in subsection 3.4.2:

Theorem 2. *We define a map* $\Delta : \mathbb{K}\,\mathsf{Sch} \to \mathbb{K}\,\mathsf{Sch} \otimes \mathbb{K}\,\mathsf{Sch}$ *by:*

$$\Delta(t) = \sum_{c \text{ pruning of } t} P^c(t) \otimes R^c(t),$$

where $R^c(t)$ *is the component of* t *containing its root and* $P^c(t) = P_1^c(t) * \cdots * P_k^c(t)$*, where* $P_i^c(t)$ *are the cut trees naturally ordered from left to right. Moreover one can write* $\Delta = \Delta_\leftarrow + \Delta_\rightarrow$ *in such a way so* $(\mathbb{K}\,\mathsf{Sch}, \prec, \cdot, \succ, \Delta_\leftarrow, \Delta_\rightarrow)$ *is a* $(3,2)$*-dendriform bialgebra.*

From the Cartier-Quillen-Milnor-Moore theorem [2], we can understand better the cocommutative part of this Hopf algebra from its primitives. Hence, a natural question is to compute the primitive elements of $\mathbb{K}\,\mathsf{Sch}$. Let us introduce the notations to reach the theorems for this purpose:

Definition 6. Let $(C, \Delta_\leftarrow, \Delta_\rightarrow)$ be a dendriform coalgebra. We define:

$$\mathrm{Prim}_\leftarrow(C) := \{c \in C \mid \tilde{\Delta}_\leftarrow(c) = 0\}, \mathrm{Prim}_\rightarrow(C) := \{c \in C \mid \tilde{\Delta}_\rightarrow(c) = 0\},$$
$$\mathrm{Prim}_{\mathrm{Coass}}(C) := \{c \in C \mid \tilde{\Delta}(c) = 0\}, \mathrm{Prim}_{\mathrm{Codend}}(C) := \{c \in C \mid \tilde{\Delta}_\leftarrow(c) = \tilde{\Delta}_\rightarrow(c) = 0\}.$$

In particular, with respect to the grading of Sch, we have:

$\mathrm{Prim}_{\mathrm{Coass}}(\mathbb{K}\,\mathsf{Sch}(1)) = \left\langle \right\rangle$, $\mathrm{Prim}_{\mathrm{Codend}}(\mathbb{K}\,\mathsf{Sch}(1)) = \left\langle \right\rangle$,

$\mathrm{Prim}_{\mathrm{Coass}}(\mathbb{K}\,\mathsf{Sch}(2)) = \left\langle \, , \, - \, \right\rangle$, $\mathrm{Prim}_{\mathrm{Codend}}(\mathbb{K}\,\mathsf{Sch}(2)) = \left\langle \right\rangle$.

Theorem 3 ([3])**.** *For all* $n \in \mathbb{N}$*, we define:*

$$\theta_n : \left\{ \begin{array}{rcl} \mathrm{Prim}_{\mathrm{Coass}}(\mathbb{K}\,\mathsf{Sch}(n)) & \to & \mathrm{Prim}_{\mathrm{Codend}}(\mathbb{K}\,\mathsf{Sch}(n+1)), \\ a \otimes \; \vee & \mapsto & a \cdot \; \vee . \end{array} \right.$$

Then, for all $n \in \mathbb{N}, \theta_n$ *is an isomorphism of vector spaces.*

Let us introduce for any $n \in \mathbb{N}$, the vector space:

$$W_n = \mathrm{Prim}_{\mathrm{Codend}}(\mathbb{K}\,\mathsf{Sch})_n \oplus \left\langle w \;\middle|\; w \in T^k(\mathrm{Prim}_{\mathrm{Coass}}(\mathbb{K}\,\mathsf{Sch}))(n) \text{ with } k > 1 \right\rangle,$$

where $T^k(V)$ is the vector space generated by words of length exactly k. From the article [15] of M. Ronco, we find an analogous of the following theorem 3 in terms of *brace algebras*. We state it in a simpler way in exchange of a loss of injectivity:

Theorem 4. *One can define a map* ω *using* $\succ, \cdot$ *and* $\prec$ *giving rise for all* $n \in \mathbb{N}$ *to:*

$$\Omega_n : \left\{ \begin{array}{rcl} T(\mathrm{Prim}_{\mathrm{Coass}}(\mathbb{K}\,\mathsf{Sch}))(n) & \twoheadrightarrow & \mathrm{Prim}_{\mathrm{Coass}}(\mathbb{K}\,\mathsf{Sch}(n)), \\ x_1 \otimes \cdots \otimes x_k & \mapsto & \omega(x_1 \otimes \cdots \otimes x_k). \end{array} \right. \tag{10}$$

This raise to a map Ω *defined over* Sch *such that* $\Omega|_{T(\mathrm{Prim}_{\mathrm{Codend}}(\mathbb{K}\,\mathsf{Sch}(n)))} = \Omega_n$*. The map* ω *is defined for any* $k \geq 2, x, y, x_1, \ldots, x_k \in \mathbb{K}\,\mathsf{Sch}$ *by:*

$$\omega(x) \coloneqq x,$$
$$\omega(x_1 \otimes \ldots \otimes x_k \otimes y) = \sum_{i=0}^{k} (-1)^{k-i} \omega_\prec(x_1 \otimes \ldots \otimes x_i) \succeq y \prec \omega_\succeq(x_{i+1} \otimes \ldots \otimes x_k),$$

where:

$$\begin{aligned}
&\omega_{\prec}(\varepsilon) = \circ = \omega_{\succ}(\varepsilon),\\
&\omega_{\prec}(x_1 \otimes \ldots \otimes x_k) = x_1 \prec \omega_{\prec}(x_2 \otimes \ldots \otimes x_k),\\
&\omega_{\succeq}(x_1 \otimes \ldots \otimes x_k) = \omega_{\succeq}(x_1 \otimes \ldots \otimes x_{k-1}) \succeq x_k.
\end{aligned}$$

Example 3. To compute an element of $\mathrm{Prim}_{\mathrm{Coass}}(\mathbb{K}\,\mathsf{Sch}(3))$, we can choose $\curlyvee \otimes \curlyvee \otimes \curlyvee$ or $\left(\curlyvee - \curlyvee\right) \otimes \curlyvee$ which are both elements of $T^{\leq 2}(\mathbb{K}\,\mathrm{Prim}_{\mathrm{Coass}}(\mathbb{K}\,\mathsf{Sch}))(3)$. Hence:

$$\begin{aligned}
\Omega_3\left(\curlyvee \otimes \curlyvee \otimes \curlyvee\right) &= \curlyvee \prec \left(\curlyvee \succeq \curlyvee\right) - \curlyvee \succeq \curlyvee \prec \curlyvee + \left(\curlyvee \prec \curlyvee\right) \succeq \curlyvee,\\
\Omega_3\left(\left(\curlyvee - \curlyvee\right) \otimes \curlyvee\right) &= -\curlyvee \prec \left(\curlyvee - \curlyvee\right) + \left(\curlyvee - \curlyvee\right) \succeq \curlyvee.
\end{aligned}$$

So, we have a recipe to produce the primitives of this algebra by induction as shown in figure 1:

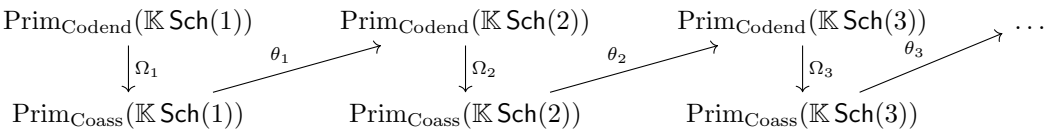


FIGURE 1. Diagram of the induction

The process of figure 1 enables us to compute $\mathrm{Prim}_{\mathrm{Coass}}(\mathbb{K}\,\mathsf{Sch})$ by induction starting with $\mathrm{Prim}_{\mathrm{Codend}}(\mathbb{K}\,\mathsf{Sch}(1)) = \left\langle \curlyvee \right\rangle$. To implement this, we find another interpretation of this problem.

3. Our point of view of Schroeder trees and their operations

Our motivation is to compute the algorithm detailed in figure 1 (for trees with at most 16 leaves otherwise, this is too huge) and to check they are primitive elements. Later, it may give hints to describe some primitives elements or at least make guesses to get an explicit combinatorial formula describing this process. But a question arise from this: "How shall we implement trees to get efficiency and an easy coding ?"

3.1. Encoding Schroeder trees.

The idea to represent Schroeder trees into a computer is to add *levels* giving rise to *levelled Schroeder trees*. From this intermediate representation, we can interpret a Schroeder tree as a binary increasing sequence which represents the levels of the tree.

3.1.1. *Levelled Schroeder Tree.*

Definition 7 ([14])**.** A *levelled Schroeder tree* is pair (t, l) where t is a Schroeder tree and for one $n \in \mathbb{N}, l : V(t) \twoheadrightarrow [\![1, n]\!]$ is a map such that for any $(u, v) \in V(t)$ where u is a descendant of v, we have $l(u) > l(v)$. For any $v \in V(t), l(v)$ is called the *level* of v.

Let $n \in \mathbb{N}$. The set of levelled Schroeder tree is denoted Sch_ℓ and the subset levelled Schroeder trees with n leaves is denoted $\mathsf{Sch}_\ell(n)$.

Remark 1. A levelled Schroeder tree is a particular case of *heap ordered trees* [9, example 2.3]. Moreover, for the sake of readability, all vertices on a same level are on a same not drawn horizontal line, see example 4.

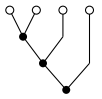
To each levelled Schroeder tree corresponds a unique Schroeder tree with a trivial surjection

$$\pi_n : \mathsf{Sch}_\ell(n) \twoheadrightarrow \mathsf{Sch}(n). \tag{11}$$

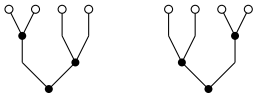
Maps π_0, π_1 and π_2 are bijective whereas π_n is not injective for $n \geq 3$.

Definition 8. Two levelled trees t and t' of $\mathsf{Sch}_\ell(n)$ are said *equivalent*, denoted $t \equiv t'$, if it represents the same Schroeder tree, that is to say, $\pi_n(t) = \pi_n(t')$.

Example 4. Referring to the example 1, except for $i = 7$, the set $\pi_3^{-1}(\{t_i\})$ has only one element. For example $\pi_3^{-1}(\{t_8\})$ consists of the unique levelled tree:



Labelled trees in $\pi_4^{-1}(\{t_7\})$ consists of the following two levelled trees:



We want to code Schroeder trees as levelled Schroeder trees. For this purpose, we need to make a choice of a representative levelled Schroeder tree of each Schroeder tree.

3.1.2. *Our point of view on Schroeder trees.*

Definition 9 (Initial chains)**.** Let $\mathcal{P} = (P, \leq)$ be a lattice with a minimum and a maximum element. Let C be a chain in this poset, we say it is *initial* if $\min(\mathcal{P})$ and $\max(\mathcal{P})$ are in the chain C.

Definition 10. Let $n \in \mathbb{N} \setminus \{0\}$. We define $\mathrm{InitChain}(n)$ the set of increasing *initial chain* of the poset $(\mathcal{P}(\llbracket 1, n \rrbracket), \subseteq)$.

Example 5. For instance, in the poset $(\mathcal{P}(\llbracket 1, 4 \rrbracket), \subseteq)$, we have the following elements of $\mathrm{InitChain}(4)$:

$$\emptyset \subseteq \llbracket 1, 4 \rrbracket, \qquad\qquad \emptyset \subseteq \llbracket 1, 3 \rrbracket \subseteq \llbracket 1, 4 \rrbracket.$$

But as said previously many levelled Schroeder trees give a same Schroeder tree. Hence, for each $n \in \mathbb{N}$ we want to build a section of π_n so that to a Schroeder tree corresponds a *unique* levelled Schroeder.

3.1.3. *Our choice of section.* We introduce the following s map to get a *normal form* of a tree:

Proposition 1. *We denote $s : \mathsf{Sch} \to \mathsf{Sch}_\ell$ the unique map such that for any $t \in \mathsf{Sch}$, $s(t)$ is the* unique *levelled Schroeder tree such that for any internal vertex of t, its level is strictly greater than all the internal vertices to its right. Then, s is well-defined and this is a section of the map* $\pi \coloneqq \bigoplus_{n=0}^{+\infty} \pi_n$.

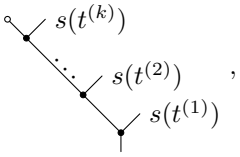
Proof. To prove s is well-defined, we detail a construction $s(t)$ by induction over the number of leaves of the tree of t. Let $t \in \mathsf{Sch}(n)$ with $n \in \mathbb{N}^*$.

Initialization: for $n = 2$, this is obvious as there exists only one representation of  as a levelled Schroeder tree.

Heredity: now let us suppose that there exists n such that for any $n' < n, t \in \mathsf{Sch}(n'), s(t)$ is well defined. We consider $t \in \mathsf{Sch}(n+1)$ and let us build $s(t)$. One can see:

$$t = t^{(1)} \vee \cdots \vee t^{(k)}$$

where for any $k \geq 2, i \in [\![1,k]\!], t^{(i)}$ is a Schroeder tree with at most n leaves. Hence, we built $s(t)$ as follows:

$s(t) =$ 

,

where for all $i \in [\![1,k]\!]$, all the vertices of $t^{(i)}$ has strictly lower levels than the root of $s\left(t^{(i+1)}\right)$. Finally by construction, we have $\pi \circ s = \mathrm{Id}_{\mathsf{Sch}}$. □

Example 6. Consider the tree

The only tree satisfying this definition among its two representatives is:

3.1.4. *The encoding.* Using the section s, we can identify any Schroeder tree with a finite list of binary words thanks to the map defined below

Definition 11. Let t be a planar tree and let $v \in V(t)$. An *angle* of t on a vertex v is a pair $(x,y) \in V(t), x \neq y$ of adjacent vertices to v such that the path from the root to v does not contain x and y, and the segment from x to y does not cross the tree.

Definition 12. Let $t \in \mathsf{Sch}$. The *height* of a tree t denoted $h(t)$ is the integer $l(s^{-1}(t))$. In other words , this the number of internal vertices of the tree.

Example 7. For instance, $h(\circ) = 0, h\left(\right.$  $\left.\right) = 1$ and $h\left(\right.$  $\left.\right) = 2$.

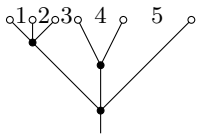
Definition 13. For any $n \in \mathbb{N}$, we introduce a map $\varphi_n : \mathsf{Sch}_\ell(n) \to \mathrm{InitChain}([\![1,n]\!])$ defined for any $t \in \mathsf{Sch}_\ell(n)$ by the *unique* initial chain $C = \left(C_0, \ldots, C_{h(t)}\right)$ describing the lifetime of the angles (labelling them from left to right by $[\![1,n]\!]$) of t in function of the levels of the tree, in other words, denoting for all $i \in [\![1,n]\!], a_i$ the i-th angle and v_i the vertex of this angle:

$$\forall i \in [\![0,h(t)]\!], C_i = \{j \in [\![1,n]\!] \,|\, l(v_j) \leq i \text{ where } v_j \text{ is the vertex of } a_j\}.$$

We also define Set_n, the map sending any element of $\mathrm{InitChain}([\![1,n]\!])$ onto a list of length n with 0's and 1's encoding the element of $\mathcal{P}([\![1,n]\!])$ considered. More precisely, for any $X \in \mathcal{P}([\![1,n]\!])$:

$$\mathrm{Set}_n(X) \coloneqq (\mathbb{1}_X(i))_{i \in [\![1,n]\!]}.$$

Example 8. For instance, reading sequences from its maximal element at the top to its minimum element at the bottom:

(12) 

$\xrightarrow{\varphi_5}$
$\begin{matrix} [\![1,5]\!] \\ [\![3,5]\!] \\ \{3\}\ \{5\} \\ \emptyset \end{matrix}$
$\xrightarrow{\mathrm{Set}_5}$
$\begin{matrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{matrix}$

Notation 2. We will call the *code map*, denoted Code, the map $\mathrm{Set} \circ \varphi \circ s : \mathsf{Sch} \to \mathrm{InitChain}$ where:

$$\mathrm{InitChain} \coloneqq \bigoplus_{n=0}^{+\infty} \mathrm{InitChain}([\![1,n]\!]).$$

Remark 2.
- The height of the tree corresponds to the number of lines needed to write the code of a tree.
- As each element of Sch_ℓ is in bijection with an initial chain of InitChain. The section s defines a normal from for initial chains that we do not detail here.

Thanks to this result we can now encode one Schroeder tree as a list of integers/sequences over $\{0,1\}$ into a computer. Now, we can look at the action of quasi-shuffles on our encoding of trees from Im(Code) for an easy computation in the free tridendriform algebra with one generator.

3.2. **The tridendriform structure on** Im(Code)**.** One usual operation we can perform on words is the concatenation. In our particular case, the tridendriform operations over InitChain are more complicated. In fact, we could multiply two initial chains with partition set of different lengths. For now on we focus on the study of levelled trees of Im(Code).‘

Remark 3. For now on, for any $n \in \mathbb{N}$, we will no longer distinguish elements of InitChain($[\![1,n]\!]$) and its image by the map Set. We also allow ourselves to mix the notations.

3.2.1. *Preliminary definitions.* In order to describe properly to the computer what we want to do, we have to identify elements encoding the comb representation of the tree associated to it.

Definition 14 (forest code)**.** Let $n \in \mathbb{N}$. Let $C' \in \mathrm{Im(Code)}$ and put $C = C' \setminus \min(C')$. Any C satisfying this property will be called a *forest code*. The *forest* associated to a forest code is the ordered concatenation of trees we get deleting the root of the tree encoded by C'.

Remark 4. The forest code associated to the empty chain, denoted ε, is $\circ$. Note that if C is a forest code that is not an initial chain, then there exists a unique forest F such that $\mathrm{Code}(T) = C'$ where T is the grafting of all the element of the forest F in this order to a common root. Hence, we extend the definition of the map Code to forests codes in such a way that $\mathrm{Code}(F) = C$.

Example 9. For instance, the forest code Code $\left(\circ\!\!\curlyvee\!\!\curlyvee\ \circ\!\!\curlyvee\ \circ\right)$ is:

$$\begin{array}{ccccc} 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 \end{array}$$

Note that Code $(\circ\ \circ\ \circ) = 1\ 1$.

Definition 15 (left comb decomposition)**.** Let $n \in \mathbb{N}$ and $C \in \mathrm{InitChain}([\![1,n]\!])$. Its *left comb decomposition*, denoted $\mathrm{LCD}(C)$, is obtained inductively the following way:

$$\mathrm{LCD}(\varepsilon) = \varepsilon,$$
$$\mathrm{LCD}(C) = (\mathrm{LCD}(C_1), F_1),$$

where in the induction we assumed $A \times (B \times C) \approx (A \times B) \times C$, and F_1 and C_1 satisfy the following block equality:

$$C = \left|\begin{array}{ccc} & 1 & \\ \tilde{C_1} & \vdots & \tilde{F_1} \\ & 1 & \\ 0\cdots0 & 0 & 0\cdots0 \end{array}\right| \text{ if it is a tree code or } C = \left|\begin{array}{ccc} & 1 & \\ \tilde{C_1} & \vdots & \tilde{F_1} \\ & 1 & \end{array}\right| \text{ if it is a forest code,}$$

such that F_1 and C_1 are respectively $\tilde{F_1}$ and $\tilde{C_1}$ keeping only once any instance of a integer in the sequence. Moreover, the red line spotted is the *leftmost* column with this property in C.

Example 10. We will start from a tree, consider its code, we apply the decomposition detailed above and compare the left comb decomposition of the tree with the forest associated to the sequence of the forest code.

$$s = \quad \longrightarrow \begin{matrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{matrix} = \mathrm{Code}(s) =: S.$$

Hence:

$$\begin{aligned} \mathrm{LCD}\,(S) &= \left(\mathrm{LCD} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \begin{matrix} 1 & 1 \\ 0 & 1 \end{matrix} \right) \\ &= \left(\varepsilon, \begin{matrix} 1 & 1 \\ 0 & 0 \end{matrix}, \begin{matrix} 1 & 1 \\ 0 & 1 \end{matrix} \right). \end{aligned}$$

Associating to it the 3-uplet of the forests code by inverting Code and applying it component-wise, we have:

$$\left(\circ, \quad , \quad \circ \right).$$

We have recovered the left comb decomposition of s after forgetting the leftmost $\circ$.

Definition 16 (right comb decomposition)**.** Let $n \in \mathbb{N}$ and $C \in \mathrm{InitChain}(\llbracket 1, n \rrbracket)$. Its *right comb decomposition*, denoted $\mathrm{RCD}\,(C)$, is obtained inductively the following way:

$$\begin{aligned} \mathrm{RCD}\,(\varepsilon) &= \varepsilon, \\ \mathrm{RCD}\,(C) &= (F_1, \mathrm{RCD}\,(C_1)), \end{aligned}$$

where in the induction we assumed $A \times (B \times C) \approx (A \times B) \times C$, and F_1 and C_1 satisfy the following block equality:

$$C = \left| \begin{matrix} \tilde{F}_1 & 1 & \tilde{C}_1 \\ & \vdots & \\ & 1 & \\ 0 \cdots 0 & 0 & 0 \cdots 0 \end{matrix} \right| \text{ if it is a tree code or } C = \left| \begin{matrix} \tilde{F}_1 & 1 & \tilde{C}_1 \\ & \vdots & \\ & 1 & \end{matrix} \right| \text{ if it is a forest code,}$$

such that F_1 and C_1 are respectively $\tilde{F}_1$ and $\tilde{C}_1$ keeping only once any instance of a integer sequence. Moreover, the red line spotted is the *rightmost* column in C ending with a 1 or with a unique 0 if the last line is full of zeros.

Example 11. We will start from a tree, consider its code, then apply the decomposition detailed above and compare the right comb decomposition of the tree with the forest associated to the

sequence of forest code.

$$t = \quad \longrightarrow \begin{matrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{matrix} = \mathrm{Code}(t) =: T$$

Hence:

$$\begin{aligned} \mathrm{RCD}\,(T) &= \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \mathrm{RCD} \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \end{pmatrix} \Bigg) \\ &= \begin{pmatrix} 1 & , & 1 & 1 \\ 0 & & 0 & 1 \end{pmatrix}, \varepsilon \Bigg) \end{aligned}$$

Associating to it the 3-uplet of the forests code, we have:

$$\left(\; , \; \circ, \circ \right).$$

We have recovered the comb decomposition of t after forgetting the rightmost $\circ$.

Definition 17. For any Schroeder tree t, we define respectively the *right tree length* and the *left tree length* by:

$$\mathrm{RTL}(t) \coloneqq |\,\mathrm{RCD}\,(t)\,| - 1 \text{ and } \mathrm{LTL}(t) \coloneqq |\,\mathrm{LCD}\,(t)\,| - 1.$$

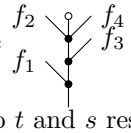
Now, we have an analogous of the comb decomposition for our sequences. We can now describe the action of a quasi-shuffle on these objects.

3.2.2. *The tridendriform structure for initial chains.* By analogy with Schroeder trees, we put:

Definition 18. Let $T \in \mathrm{InitChain}([\![1, n]\!])$ and $S \in \mathrm{InitChain}([\![1, m]\!])$. Let us denote $k = \mathrm{RTL}(T)$ and $l = \mathrm{LTL}(S)$. Consider $\sigma \in \mathrm{QSh}(k, l)$ which has for image $[\![1, r]\!]$. We denote by $\sigma(T, S)$ the element of $\mathrm{InitChain}([\![1, m + n]\!])$ defined by:

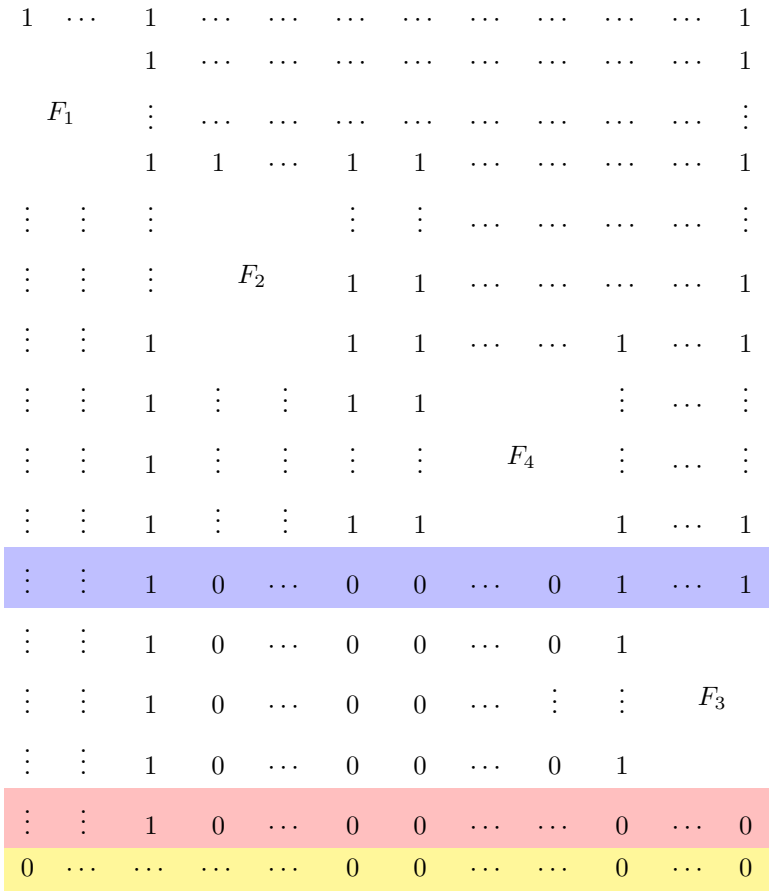
(1) on the first m integers, copy all the k forests codes of $\mathrm{RCD}\,(T)$ (excepted to the rightmost one $\circ$) separated by a column of ones on their right;
(2) for h from r to 1 with a -1 step:
   Case 1: if $\sigma^{-1}(\{h\}) = \{i\}, i \in [\![1, k]\!]$, write 0's below the code of F_i and on its adjacent right column.
   Case 2: if $\sigma^{-1}(\{h\}) = \{j\}, j \in [\![k + 1, k + l]\!]$, include the code of F_j at its appropriate place shifted by n bits with a ones column on its left. Write 0's below the code of F_j and on its adjacent left column.
   Case 3: if $\sigma^{-1}(\{h\}) = \{i, j\}, (i, j) \in [\![1, k]\!] \times [\![k + 1, k + l]\!]$, include the code of F_j at its appropriate place shifted by n bits with a ones column on its left. Then, write 0's below the code of F_j and on its adjacent left column *and* below the code of F_i and on its adjacent right column.

   The element obtained from this operation is denoted $\sigma(T, S)$.

Example 12. Consider the $(2,2)$-quasi-shuffle $\sigma = (1,3,2,3)$. Let us take $t =$  and $s =$  . Then $\sigma(t,s) =$  . Then, let us denote for all $i \in [\![1,4]\!]$, $\mathrm{Code}(f_i) = F_i$. Hence, the code associated to t and s respectively denoted T and S are of the shape:

$$T = \begin{array}{cccccc} 1 & \cdots & 1 & 1 & \cdots & 1 \\ & & 1 & 1 & \cdots & 1 \\ & F_1 & \vdots & 1 & \cdots & 1 \\ & & 1 & 1 & \cdots & 1 \\ \vdots & \vdots & 1 & & & 1 \\ \vdots & \vdots & 1 & F_2 & & \vdots \\ \vdots & \vdots & 1 & & & 1 \\ \vdots & \vdots & 1 & 0 & \cdots & 0 \\ 0 & \cdots & 0 & 0 & \cdots & 0 \end{array} \quad \text{and } S = \begin{array}{cccccc} 1 & \cdots & 1 & 1 & \cdots & 1 \\ 1 & & & 1 & \cdots & 1 \\ \vdots & & F_4 & & 1 & \cdots & 1 \\ 1 & & & 1 & \cdots & 1 \\ 0 & \cdots & 0 & 1 & \cdots & 1 \\ 0 & \cdots & 0 & 1 & & \\ 0 & \cdots & 0 & \vdots & F_3 & \\ 0 & \cdots & 0 & 1 & & \\ 0 & \cdots & 0 & 0 & \cdots & 0 \end{array}$$

Then, $\sigma(T,S)$ is the code of the tree $\sigma(t,s)$ given by:



With this definition, we are now able to define three operators in Im(Code)

Definition 19. Let T and S be two elements in Im(Code) such that $k = |\operatorname{RTL}(T)|$ and $l = |\operatorname{LTL}(S)|$. We define:

$$T \prec S = \sum_{\substack{\sigma \in \mathrm{QSh}(k,l) \\ \sigma^{-1}(\{1\}) = \{1\}}} \sigma(T,S), \quad T \succ S = \sum_{\substack{\sigma \in \mathrm{QSh}(k,l) \\ \sigma^{-1}(\{1\}) = \{1\}}} \sigma(T,S),$$

$$T \cdot S = \sum_{\substack{\sigma \in \mathrm{QSh}(k,l) \\ \sigma^{-1}(\{1\}) = \{1,k+1\}}} \sigma(T,S).$$

In the sequel, we show that this structure is a tridendriform structure on Im(Code) showing it is isomorphic to the free tridendriform algebra of Schroeder trees. Hence it gives us a mathematical background to implement programs in section 4.

Theorem 5. *Let t, s be two Schroeder trees with shapes given in equation* (1)*. Hence, k is the right comb length of t and l the left comb length of s. Let $\sigma \in \mathrm{QSh}(k,l)$. Then:*

$$\mathrm{Code}(\sigma(t,s)) = \sigma(\mathrm{Code}(t), \mathrm{Code}(s)).$$

Proof. In order to prove this theorem, we will proceed by induction over the sum of k and l. If $l = 0$ or $k = 0$, this means that $s = \circ$ or $t = \circ$. We initialize the induction for $0, 1$ and 2.

Initialization: if $\max(k,l)=1$ and $k+l=1$, then the identity shuffle is acting on this pair of tree and one of them is $\circ$. By definition 18, we get back the tree code of the non-unit element. Otherwise, if $k+l=2$ and $k,l\neq 0$ hence $\sigma=(1,1)$, then:

$$t=\ f_1\ \curlyvee\ ,\ s=\ \curlyvee\ f_2\quad\text{and}\quad \sigma(t,s)=\ f_1\ \curlyvee\ f_2$$

Denoting $\mathrm{Code}(f_1)=F_1$ and $\mathrm{Code}(f_2)=F_2$, their respective codes are:

$$T=\begin{array}{ccc}1&\cdots&1\\&&1\\&F_1&\vdots\\&&1\\0&\cdots&0\end{array},\ S=\begin{array}{ccc}1&\cdots&1\\1&&\\&F_2&\\1&&\\0&\cdots&0\end{array}\quad\text{and}\quad \sigma(T,S)=\begin{array}{ccccccccc}1&\cdots&1&1&1&1&\cdots&1\\&&&1&1&1&\cdots&1\\&F_1&&\vdots&\vdots&\vdots&\vdots&\vdots\\&&&1&1&1&\cdots&1\\0&\cdots&0&1&1&&&\\0&\cdots&0&\vdots&\vdots&&F_2&\\0&\cdots&0&1&1&&&\\0&\cdots&0&0&0&0&\cdots&0\end{array}$$

Note that the code of $\sigma(T,S)$ has for associated tree $\sigma(t,s)$. If $k=0$ or $l=0$, then the result is obvious.

Heredity: Suppose that there exists $r\geq 1\in\mathbb{N}^*$ such that for all couple (k,l) with $k+l\leq r$, we have for any t,s whose tree length are respectively k and l:

$$\sigma(\mathrm{Code}(t),\mathrm{Code}(s))=\mathrm{Code}(\sigma(t,s)).$$

Let us prove that this property is true for the next rank. Let t,s be two trees with right comb length k and left comb length l, let $\sigma\in\mathrm{Sh}(k,l)$ such that $k+l=r+1$. Note that if $k=0$ or $l=0$, the result is obvious by definition 19. There are three possible cases left, in those cases, we use the notations of the proof of lemma 1:

First case: $\sigma^{-1}(\{1\})=\{1\}$: Then:

$$\mathrm{Code}(\sigma(t,s))=\mathrm{Code}\left(\ f_1\ \curlyvee\ \sigma'(t',s)\ \right)\ \text{where } t'=\ f_1\ \curlyvee\ t'\ ,$$

$$=\begin{array}{ccccccc}1&\cdots&1&1&\cdots&1\\&&1&1&\cdots&1\\&F_1&\vdots&1&\cdots&1\\&&1&1&\cdots&1\\\vdots&\vdots&1&&&\\\vdots&\vdots&\vdots&\multicolumn{3}{l}{\mathrm{Code}(\sigma'(t',s))}\\\vdots&\vdots&1&&&\\0&\cdots&0&0&\cdots&0\end{array}$$

Applying the induction hypothesis because $\mathrm{RTL}(t') + \mathrm{LTL}(s) \leq r$, we have:

$$\mathrm{Code}(\sigma(t,s)) = \begin{array}{ccccccc} 1 & \cdots & 1 & 1 & \cdots & & 1 \\ & & & 1 & 1 & \cdots & 1 \\ & F_1 & & \vdots & 1 & \cdots & 1 \\ & & & 1 & 1 & \cdots & 1 \\ \vdots & & \vdots & 1 & & & \\ \vdots & & \vdots & \vdots & \sigma'(\mathrm{Code}(t'), \mathrm{Code}(s)) & & \\ \vdots & & \vdots & 1 & & & \\ 0 & \cdots & 0 & 0 & \cdots & & 0 \end{array}$$

$$= \sigma(\mathrm{Code}(t), \mathrm{Code}(s)),$$

by definition of the action of σ over a pair of forests codes.

Second case: $\sigma^{-1}(\{1\}) = \{k+1\}$ is similar to the first one. So it is left to the reader.

Third case: $\sigma^{-1}(\{1\}) = \{1, k+1\}$. Then:

$$\mathrm{Code}(\sigma(t,s)) = \mathrm{Code}\left(\begin{array}{c} \sigma'(t',s') \\ f_1 \quad\quad f_{k+1} \end{array} \right) \text{ where } s = \begin{array}{c} s' \\ \quad f_{k+1} \end{array}$$

$$= \begin{array}{ccccccccccc} 1 & \cdots & 1 & 1 & 1 & \cdots & 1 & 1 & 1 & \cdots & 1 \\ & & & \vdots & \vdots & \cdots & \vdots & \vdots & \vdots & \cdots & \vdots \\ & F_1 & & 1 & 1 & \cdots & 1 & 1 & 1 & \cdots & 1 \\ & & & 1 & 1 & \cdots & 1 & 1 & 1 & \cdots & 1 \\ 0 & \cdots & 0 & 1 & & & & \vdots & \vdots & \cdots & \vdots \\ \vdots & & \vdots & \vdots & \mathrm{Code}(\sigma'(t',s')) & & & 1 & 1 & \cdots & 1 \\ 0 & \cdots & 0 & 1 & & & & 1 & 1 & \cdots & 1 \\ 0 & \cdots & 0 & 1 & 0 & \cdots & 0 & 1 & & & \\ \vdots & & \vdots & \vdots & 0 & \cdots & \vdots & \vdots & & F_{k+1} & \\ 0 & \cdots & 0 & 1 & 0 & \cdots & 0 & 1 & & & \\ 0 & \cdots & 0 & 1 & 0 & \cdots & 0 & 1 & 0 & \cdots & 0 \\ 0 & \cdots & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 \end{array}$$

Applying the induction hypothesis as $\mathrm{RTL}(t')+\mathrm{LTL}(s') \leq r$, we have $\mathrm{Code}(\sigma'(t',s')) = \sigma(\mathrm{Code}(t'), \mathrm{Code}(s'))$. Then applying the construction case number 3 in definition 18, one has:

$$\mathrm{Code}(\sigma(t,s)) = \sigma(\mathrm{Code}(t), \mathrm{Code}(s)).$$

Hence by the induction principle, we have proved the theorem. □

Finally, the map Code is also linear hence:

Corollary 1. *The* Code *map is an isomorphism of tridendriform algebras into its image.*

Proof. To prove this corollary, one just needs to show that Code is a tridendriform morphism from $(\mathsf{Sch}, \prec, \succ, \cdot)$ into $(\mathrm{Im}(\mathrm{Code}), \prec, \succ, \cdot)$. Let $t, s \in \mathsf{Sch}$ such t has right comb length k and s has left comb length l. We show Code is a morphism for the middle product:

$$\begin{aligned}\mathrm{Code}(t \cdot s) &= \sum_{\substack{\sigma \in \mathrm{QSh}(k,l) \\ \sigma^{-1}\{1\} = \{1, k+1\}}} \mathrm{Code}(\sigma(t, s)) \\ &= \sum_{\substack{\sigma \in \mathrm{QSh}(k,l) \\ \sigma^{-1}\{1\} = \{1, k+1\}}} \sigma(\mathrm{Code}(t), \mathrm{Code}(s)) \\ &= \mathrm{Code}(t) \cdot \mathrm{Code}(s).\end{aligned}$$

The other cases are similar. □

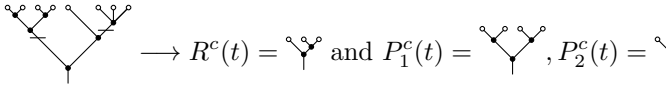
Now that we are sure the two descriptions are isomorphic. Before getting into coding, we describe what is a cut on our codes and its relationship with some *packed words*.

3.3. **Short description of the coproduct.** We now describe without much details how to interpret the coproduct operation of theorem 2 for tree codes. Let us remind what is an admissible cut:

Definition 20. Let t be a Schroeder tree. An *pruning* of t is a choice of internal edges in t such that any path from a leaf of the tree to its root meets at most one chosen edge. A pruning is called a *single cut* when it chooses only one edge. Hence, a pruning can be composed of many single cuts. Given a pruning c, it splits the tree into many connected components removing cut edges. The component with the root of the starting tree is denoted $R^c(t)$ and the other components are denoted $P_1^c(t), \ldots, P_k^c(t)$. We also consider both elements as prunings:

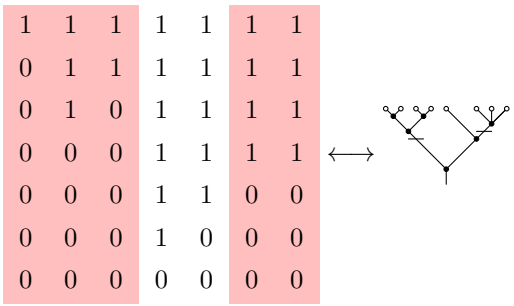
- the *empty cut* giving $R^c(t) = t$ and $P^c(t) = \circ$;
- the *total cut* giving $R^c(t) = \circ$ and $P^c(t) = t$.

Example 13. For instance, symbolizing a cut edge by an horizontal line, one has:



In other words, a pruning consists of withdrawing an interval of angles such that both adjacent angles (if they exist) are still living, otherwise the pruning is not admissible. Translating this in term of tree codes, doing a pruning of a tree t with l angles is choosing a subset of $[\![1, l]\!]$ where for each integer interval of this subset, when the tree code restricted to this subset is 0 for the first time at line i and the two adjacent columns has a one at line i.

Example 14. For instance, let us consider the tree code of the tree in example 13, the subset of angles corresponds to the cut:



3.4. **Links with packed words.**

3.4.1. *Description of these.*

Definition 21. We define LPPW the subset of the set of packed words PW called *Left Priority Packed Word*. This set is generated by the empty packed word denoted ε with a construction rule $c(w_1, \dots, w_k)$ where $w_1, \dots, w_k \in \text{LPPW}$ by:

$$c(w_1, \dots, w_k) = w_1 \ N \ (w_2 + \max(w_1)) \ N \dots N \ \left(w_k + \sum_{i=1}^{k-1} \max(w_i) \right)$$

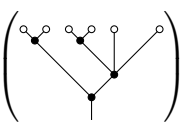
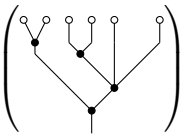
where $N = \sum_{i=1}^{k} \max(w_i) + 1$.

Example 15. Consider $w_1 = 132$ and $w_2 = 211$, hence $c(w_1, w_2) = 1326544$.

Consider $t \in \mathsf{Sch}_\ell, n = h(t)$ and its enumeration of angles $(a_i)_{i \in [\![1,k]\!]}$. We define a map $\tilde{W} : \mathsf{Sch}_\ell \to \text{PW}$ mapping t to a packed word $w = w_1 \dots w_k$ where w_i is $n - l(v_i)$ where v_i is the vertex of the angle a_i. We also define $W : \mathsf{Sch} \to \text{PW}$ by $\tilde{W} \circ \pi$.

Remark 5. The map $\tilde{W}$ is the *snake walk* of a levelled Schroeder tree.

Example 16. For instance:

$\tilde{W}$  $= 13122$ and W  $= 14233$.

It turns out Schroeder trees are in bijection with LPPW:

Proposition 2. *The languages* $(\diamond, \vee)$ *and* (ε, c) *are isomorphic. Hence,* W *is a bijection between* LPPW *and* Sch .

Proof. One just needs to see that $\vee$ and c are both injective construction rules. □

3.4.2. *Enumeration of prunings in* LPPW.

Definition 22. We define *prunings* of an element $w \in \text{LPPW}$ denoted $P(w)$ using induction over LPPW:

$$P(\emptyset) = \{\emptyset\}, \qquad P(c(w_1, \dots, w_k)) = P(w_1) \times P(w_2) \times \dots \times P(w_k) \cup \{\text{tot}\}.$$

where tot symbolizes the total cut.

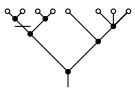
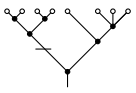
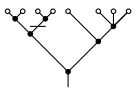
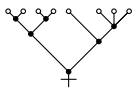
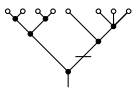
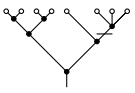
This corresponds to the natural prunings for trees, however this definition is not really kind to use. To get a nicer definition, we enumerate single cuts of left priority packed word.

Definition 23. A *single cut* of $w \in \text{LPPW}$ is a factor (i.e a sequence of consecutive letters of w) where adjacent letters of this factor are strictly greater than all the letters of this factor.

Noticing than a single cut deletes an integer interval of angles, we deduce a procedure to enumerate every single cuts of a tree t with k angles given $W(t) = w \in \text{LPPW}$:

- for $i \in [\![1, k]\!]$, get the smaller factor f of w containing i;
- from this factor, gets the angles associated to the position of the original letters in w;
- this factor f corresponds to the *unique* single cut of t withdrawing this list of angles.

Example 17. Consider the Schroeder tree of Example 13 whose image by W is 1326544. Hence, applying the procedure detailed above, one gets 6 simple cuts according to the next table

Angle	1	2	3	4	5	6	7
Factors	1	132	2	1326544	544	44	44
Cut angles	1	132	4	$[\![1, 7]\!]$	567	67	67
Single cut							

It defines properly how to do a pruning of an element of LPPW as a pruning is a sequence of non-intersecting single cuts.

4. Implementing the problem

Thanks to the map denoted B, any sequence with 0's and 1's is identified to an integer the following way:

$$\forall s \in \{0,1\}^\star, B(s) = \sum_{i=0}^{|s|} s_i \cdot 2^i.$$

Hence, in our code, we will manipulate such sequences as integers. Then, we have to build enumerations and structures needed to perform computations:

- enumeration of quasi-shuffles;
- class describing Schroeder trees as sequences of 0's and 1's;
- the Ω map;
- Schroeder trees enumeration;
- the cut coproduct Δ.

We do briefs comments about how this code is made. For our objective, we only implemented Schroeder trees with at most 16 leaves as they are at this point too much elements (10 463 578 353 trees and 745 387 038 codendriform primitives) to see anything. We have made the choice to implement this in `C++` to get a faster execution code than one done with `Python` or `Sagemath`.

4.1. **Quasi-shuffle enumeration.** In the code available on GitHub [5], one can find a procedure `Quasishuffle` that returns the list of all quasi-shuffle of a given type (n, m). For this, we use the result from lemma 1 to get an inductive algorithm computing all quasi-shuffles of a given type.

4.2. **The class Schroeder tree.** We define two classes `SchroederTree` and `SchroederForest` where:

(1) a tree is list of 16 integers between 0 and 2^{16}. An integer of this lists represents a sequence of zeros and ones needed to build a Schroeder tree as explained. Hence, its first element is $2^{l+1} - 1$ where l is the number of angles of the tree and the last one is 0.
(2) a forest is as well a list

4.3. **The Ω map.** The map Ω detailed in theorem 4 is simplified compared to the one given by M. Ronco [15] to the price of non-injectivity to get an easier coding.

Let $n \in \mathbb{N}$ and suppose we know $\mathrm{Prim}_{\mathrm{Coass}}(k)$ for all $n < k$ and $\mathrm{Prim}_{\mathrm{Codend}}(n)$. In order to compute $\mathrm{Prim}_{\mathrm{Coass}}(n)$, we proceed the following way:

- we generate an ordered partition of $n = a_1 + \cdots + a_l$ of length l , denoted $\mathbf{a}$, for all $l \in [\![1, n]\!]$;
- if $\mathbf{a} = (n)$, then we apply the identity map;

- else according to this partition $\mathbf{a} = (a_1, \ldots, a_l)$ with $l \geq 2$, we choose one element e_i in each space $\mathrm{Prim}_{\mathrm{Coass}}(a_i)$ exhaustively and compute $\Omega(a_1 \otimes a_2 \otimes \ldots \otimes a_l)$;
- we put in memory the results in a matrix, fixing an enumeration of Schroedertrees of degree n. Columns are indexed by this enumeration and the ith row is the ith result obtained by this algorithm;
- Once the enumeration of ordered partitions is finished, we compute a generating family of the image space of this matrix that we can display.

Some computations following this process are given as appendices.

4.4. **Schroeder enumeration.** Given $n \in \mathbb{N}$, we provide an enumeration of $\mathsf{Sch}(n)$ thanks to the *snake walk* of a tree since doing this walk tree maps it to a packed word in a bijective way which is easily enumerable the following way

4.5. **The cut coproduct.** The map $\Delta : \mathbb{K}\,\mathsf{Sch} \to \mathbb{K}\,\mathsf{Sch} \otimes \mathbb{K}\,\mathsf{Sch}$ detailed in theorem 2 is coded the following way:

- for a given tree code C, get all its single cuts using the algorithm of section 3.4.2;
- then using a pile, enumerate all the possible prunings;
- for any pruning, compute the term of the coproduct associated to it using levels truncation.

References

[1] Sage documentation.

[2] Pierre Cartier and Frédéric Patras. *Classical Hopf algebras and their applications*, volume 29 of *Algebra and Applications*. Springer, Cham, [2021] ©2021.

[3] Pierre Catoire. Tridendriform structures. *Symmetry, Integrability and Geometry: Methods and Applications*, September 2023.

[4] Pierre Catoire, Pierre Clavier, and Douglas Modesto da Fraga Candido. Tridendriform and dendriform Zeta Values from Schroeder trees. working paper or preprint, August 2025.

[5] Pierre Catoire and Jean Fromentin. 2025.

[6] Adrian Celestino, Kurusch Ebrahimi-Fard, Frédéric Patras, and Daniel Perales. Cumulant-cumulant relations in free probability theory from Magnus' expansion. *Found. Comput. Math.*, 22(3):733–755, 2022.

[7] Fréderic Chapoton. Free dendriform algebras, 2017.

[8] Kurusch Ebrahimi-Fard and Frédéric Patras. Monotone, free, and boolean cumulants: a shuffle algebra approach. *Adv. Math.*, 328:112–132, 2018.

[9] Robert Grossman and Richard G. Larson. Hopf-algebraic structure of families of trees. *J. Algebra*, 126(1):184–210, 1989.

[10] M. E. Hoffman. Quasi-shuffle products. *J. Algebraic Combin.*, 11:49–68, 2000.

[11] Jean-Louis Loday and María O. Ronco. Hopf algebra of the planar binary trees. *Adv. Math.*, 139(2):293–309, 1998.

[12] J. Peter May. Some remarks on the structure of Hopf algebras. *Proc. Amer. Math. Soc.*, 23:708–713, 1969.

[13] John W. Milnor and John C. Moore. On the structure of Hopf algebras. *Ann. of Math. (2)*, 81:211–264, 1965.

[14] Patricia Palacios and María O. Ronco. Weak Bruhat order on the set of faces of the permutohedron and the associahedron. *J. Algebra*, 299(2):648–678, 2006.

[15] María Ronco. Eulerian idempotents and Milnor-Moore theorem for certain non-cocommutative Hopf algebras. *J. Algebra*, 254(1):152–172, 2002.

[16] Yi Zhang and Xing Gao. Hopf algebras of planar binary trees: an operated algebra approach. *Journal of Algebraic Combinatorics*, 51(4):567–588, May 2019.

[17] Yuanyuan Zhang, Xing Gao, and Dominique Manchon. Free rota-baxter family algebras and free (tri)dendriform family algebras, 2020.

[18] Yuanyuan Zhang, Xing Gao, and Dominique Manchon. Free (tri)dendriform family algebras. *J. Algebra*, 547:456–493, 2020.

[19] Paul Zimmermann, Alexandre Casamayou, Nathann Cohen, Guillaume Connan, Thierry Dumont, Laurent Fousse, François Maltey, Matthias Meulien, Marc Mezzarobba, Clément Pernet, Nicolas M. Thiéry, Erik Bray, John Cremona, Marcelo Forets, Alexandru Ghitza, and Hugh Thomas. *Computational Mathematics with SageMath*. 2018.